Keywords: graph neural networks • Muon optimizer • second-order methods • graph-aware preconditioning

Module-Aware Optimization for Graph Neural Networks

GraphMuon Adapts the Muon Optimizer to Graph-Propagated Input Covariance, Achieving 2.13–7.38× Wall-Clock Speedups Over AdamW

By Guy Hadad, Haggai Roitman, Moshe Eliasof, Bracha Shapira
arXiv:2611.08379 • cs.LG / cs.AI • NeurIPS 2026

The Muon optimizer orthogonalizes gradient matrices to precondition updates, but its implicit geometry ignores graph propagation, which systematically distorts layer-input covariance in GNNs. GraphMuon corrects this by orthogonalizing in the graph-propagated input metric, cutting time-to-target by up to 7.38× over AdamW on 15 benchmark configurations.

AT A GLANCE

Problem: Muon's matrix update assumes isotropic input covariance, but graph propagation distorts covariance, amplifying updates and destabilizing GNN training.

Why it matters: GNNs are increasingly used for large-scale scientific and industrial graph tasks; faster, stable optimization directly reduces training cost.

Method: Replace Muon's Euclidean metric with the graph-propagated input covariance; orthogonalize and normalize step sizes in this graph-aware metric.

Result: 2.13–7.38× wall-clock speedup over AdamW on 13/15 dataset-architecture configs; vanilla Muon diverges on 4/15.

Evidence: 15 configurations across node classification, inductive graph classification, and graph regression benchmarks.

The Big Picture

to AdamW for large neural network training. Its core mechanism is matrix orthogonalization: rather than scaling the gradient element-wise (as Adam does), Muon computes the polar decomposition of the gradient matrix and uses the orthogonal factor as the update. This is equivalent to preconditioned gradient descent with preconditioner $(GG^T)^{-1/2}$, implicitly implementing a form of second-order optimization using only the gradient's own geometry.

Graph neural networks break this assumption. In a GNN layer, the input to each linear transformation is the aggregated neighborhood feature $H = \bar{A}X$, where $\bar{A}$ is the normalized adjacency operator. Graph propagation concentrates variance in directions corresponding to low-frequency graph components and suppresses high-frequency directions. Applying Muon's Euclidean orthogonalization ignores this distortion, causing the update to be under-calibrated in smooth directions and over-calibrated in rough directions—destabilizing training in poorly conditioned graphs.

Why Existing Approaches Fall Short

AdamW applies element-wise adaptive scaling, which is permutation-invariant and does not exploit the matrix structure of weight gradients. It is robust but slow to converge on tasks where curvature is anisotropic along structured directions.

Vanilla Muon captures matrix structure but uses a Euclidean metric that is mismatched to graph-propagated inputs. On graphs with high spectral anisotropy (e.g., large homophilic graphs with strong community structure), the covariance of H is far from isotropic, and Muon's updates amplify variance along over-represented directions, causing divergence.

SOAP and Shampoo are full Kronecker-factored second-order methods that in principle handle anisotropic curvature, but their cost per step is cubic in the layer dimension, making them impractical for large GNN layers. GraphMuon achieves the efficiency of Muon (one Newton-Schulz iteration per step, $O(d^2)$ cost) while correcting for the GNN-specific covariance structure.

Core Mechanism

For a GNN layer with propagated input $H = \bar{A}X \in R^{n\times d}$ and weight matrix $W \in R^{d'\times d}$, the empirical input covariance is $\Sigma = H^TH / n$.

Muon computes the polar factorization $G = UP$ and uses U as the update (orthogonalization in the standard Frobenius metric). GraphMuon instead orthogonalizes in the Σ-weighted metric $\langle A,B\rangle_\Sigma = tr(A\Sigma B^T)$. The Σ-orthogonal factor of G is:

$$G\Sigma^{1/2} = \bar{U}\hat{P},\ U_\Sigma = \bar{U}\Sigma^{-1/2}$$

The step-size normalization replaces the spectral-norm factor used in Muon with $\eta_\Sigma = \eta / ||U_\Sigma H^T||_F$, ensuring the effective learning rate measures actual change in layer output.

covariance ($\beta = 0.95$). The GraphMuon update at step t is $W_{t+1} = W_t - \eta_\Sigma U_{\Sigma_t}$, where U_{Σ_t} is computed via two Newton-Schulz iterations applied to $G_t \Sigma_t^{1/2}$:

$$B_0 = G_t \Sigma_t^{1/2}, \quad B_{k+1} = (3/2)B_k - (1/2)B_k B_k^T B_k, \quad U_{\Sigma_t} = B_2 \Sigma_t^{-1/2}$$

Per-step overhead over vanilla Muon is dominated by the matrix square roots and is bounded by $O(d^3 / n_{steps})$ amortized.

Learning or Inference Procedure

GraphMuon is a drop-in replacement for AdamW or Muon in any GNN training pipeline. Required inputs are gradient matrices G_t for each GNN weight matrix, propagated activations H_t for each layer (cached from the forward pass), an EMA coefficient $\beta = 0.95$, and two Newton-Schulz iterations. No separate profiling or warm-up phase is required; the EMA initializes from the first mini-batch and stabilizes within a few hundred steps.

What the Guarantee Says

No formal convergence rate is derived. The empirical claim is: on 13 of 15 evaluated dataset-architecture configurations, GraphMuon reaches a fixed performance target faster (in wall-clock time) than AdamW. On the remaining 2, it is within 5% of AdamW's time-to-target. Vanilla Muon fails to converge on 4 of the 15 configurations; GraphMuon converges on all 15.

Experimental Findings

- **Wall-clock speedup:** 1.3–2.1× faster than AdamW on 13/15 configs; speedups are largest on dense homophilic graphs.
- **Peak accuracy:** Marginally higher than AdamW in 9/15 configs; never more than 0.3% lower in any configuration.
- **Vanilla Muon failures:** Muon diverges on 4/15 configs (ogbn-arxiv with GraphSAGE, ZINC with GNN+, and 2 TUDatasets); GraphMuon converges on all.
- **Overhead:** Per-step overhead over AdamW is 8–12% in wall-clock time; the speedup from fewer steps more than compensates.

Ablations and Interpretation

EMA vs. online covariance: Using a fresh covariance estimate per step achieves the same quality but at 2.3× the per-step cost; EMA captures the same effective geometry at lower cost.

Newton-Schulz iterations: One iteration is insufficient for high spectral ratio Σ; two iterations yield near-exact Σ-orthogonalization in all evaluated cases.

Step-size normalization: Replacing the output-space normalization with the standard Muon spectral-norm normalization degrades time-to-target by 18% on average.

Covariance estimation noise: On sparse graphs (fewer than 100 nodes per batch), the EMA estimate is noisy; adding a 5% identity regularizer to Σ_t recovers the full speedup without hurting peak accuracy.

Reference

Guy Hadad, Haggai Roitman, Moshe Eliasof, Bracha Shapira. **Module-Aware Optimization for Graph Neural Networks.** arXiv:2611.08379. NeurIPS 2026. arxiv.org/abs/2611.08379